PAI-4 DevSecOps — Planteamiento de la solución (ST20)

Nov 13, 2026 · @Fran

1. Qué nos piden

Montamos un pipeline DevSecOps en GitHub Actions sobre OWASP Juice Shop, con 4 tipos de análisis (SCA, SAST, IaC y DAST), quality gates y envío automático de todo a DefectDojo. Entrega el **2 de diciembre de 2026 a las 23:59** como **PAI4-ST20.zip**.

El cliente (una empresa de desarrollo web) ha tenido incidentes porque su pipeline CI/CD no comprueba la seguridad del código, de las dependencias ni de la configuración, y además está migrando a microservicios en contenedores sobre Kubernetes. Trabajamos como Yellow Team: quien construye el software automatiza también sus pruebas de seguridad.

Objetivo	Qué hay que demostrar
1. Diseñar y montar el pipeline	Diagrama del pipeline (1.a) y diagrama de flujo ante una vulnerabilidad (1.b), coherentes con los ficheros del pipeline
2. ⩾ 3 herramientas de tipos distintos	Elección justificada según la tecnología de la app
3. Integración automática	Se ejecutan en cada push / pull request, cada una en su etapa
4. Quality gates	Cada herramienta detecta ⩾ 1 vulnerabilidad; hay etapas en modo aviso y al menos un gate que bloquea de verdad
5. Gestión centralizada	Resultados enviados por API a DefectDojo: deduplicar, clasificar, priorizar

Entregable (PAI4-ST20.zip): informe técnico PDF (máx. 15 páginas sin portada ni anexos, con los dos diagramas y el enlace al repositorio), código y configuración del pipeline, y evidencias (logs, al menos una ejecución que falla por un quality gate y los hallazgos en DefectDojo).

Fechas: seguimientos el 17/18 y el 24/25 de noviembre; entrega el 2 de diciembre. Penalización del 10 % por día de retraso.

Reglas que penalizan: credenciales en el repositorio (la API key de DefectDojo va como secreto del pipeline) y escanear algo que no sea nuestra propia aplicación desplegada.

2. Decisiones de diseño

Elegimos Juice Shop + GitHub Actions + DefectDojo porque es la combinación con más soporte nativo: todas las herramientas tienen acción oficial de GitHub o imagen Docker y un parser propio en DefectDojo.

Aplicación: OWASP Juice Shop. Node.js (Express) + Angular/TypeScript, trae Dockerfile y tiene vulnerabilidades en las cuatro capas que vamos a analizar: código propio (SQLi, XSS, secretos), dependencias antiguas a propósito, configuración de contenedor y fallos en ejecución (cabeceras, CORS, cookies). Además su Dockerfile genera una imagen que podemos escanear y desplegar, lo que encaja con el escenario de contenedores del cliente. WebGoat (Java) obligaría a Maven y a herramientas Java más lentas; DVWA no tiene casi dependencias para el SCA.

CI/CD: GitHub Actions. El curso ya usa GitHub, no hace falta montar servidor (Jenkins) y `continue-on-error` da el modo aviso que pide el enunciado. Repositorio público: minutos de Actions ilimitados y el profesorado puede verlo sin invitación.

Herramientas (4 tipos, el mínimo es 3):

Tipo	Herramienta	Por qué esta y no otra	Informe	Parser en DefectDojo
SCA	OWASP Dependency-Check	Libre, analizador específico de Node (package-lock / npm audit) y base de datos NVD; `--failOnCVSS` da el umbral directamente	XML + HTML	Dependency Check Scan
SAST	Semgrep Community Edition	Reglas mantenidas para JavaScript/TypeScript y Express (`p/owasp-top-ten` , `p/nodejsscan` , `p/secrets`); rápido (< 2 min) y sin servidor, a diferencia de SonarQube	JSON	Semgrep JSON Report

Tipo	Herramienta	Por qué esta y no otra	Informe	Parser en DefectDojo
IaC	Checkov	Analiza Dockerfile y manifiestos de Kubernetes en una sola pasada; cada fallo trae severidad e ID (`CKV_K8S_*`, `CKV_DOCKER_*`)	JSON	Checkov Scan
Contenedor (SCA de imagen)	Trivy image	Complementa al SCA: ve los paquetes del sistema de la imagen base y los `node_modules` empaquetados en la imagen que realmente se desplegaría	JSON	Trivy Scan
DAST	OWASP ZAP (acciones oficiales de GitHub)	Imagen Docker y acción oficial; ajax spider para una SPA en Angular; fichero de reglas para marcar falsos positivos	XML + HTML	ZAP Scan
Gestión	DefectDojo (Community)	Recomendado en el enunciado, API v2 con `reimport-scan`, deduplicación por hash y parsers para todas las herramientas anteriores	—	—

IAST queda fuera: las herramientas son comerciales y no aporta nada que no cubran SAST + DAST. CodeQL y Dependabot se pueden activar gratis en el repo público como extra, pero no cuentan como etapas del pipeline.

Kubernetes: escribimos nosotros los manifiestos (`k8s/`) de despliegue de Juice Shop, sin endurecer, para reproducir el escenario del cliente y que Checkov tenga qué detectar. El despliegue final se hace en un clúster efímero `kind` dentro del propio runner.

3. Repositorio y entorno

Un repositorio nuevo y público, `ST20-PAI4-DevSecOps`, con el código de Juice Shop en la raíz (copiado de una release fijada, no un fork) y nuestro pipeline encima. DefectDojo corre fuera de GitHub, en una máquina del grupo, expuesto por un túnel HTTPS.

Por qué un repo aparte y no una carpeta de `Frarodmar19/SSII`: GitHub Actions solo lee los workflows de `.github/workflows/` en la raíz, y el Dockerfile de Juice Shop espera su código como contexto de build. Un fork arrastraría los workflows del proyecto original, que hay que borrar igualmente. En el repo de la asignatura se deja un enlace en `PAI4/README.md`.

```
ST20-PAI4-DevSecOps/
├── .github/
│   ├── workflows/devsecops.yml
│   └── dependabot.yml
├── k8s/
│   ├── namespace.yaml
│   ├── deployment.yaml
│   └── service.yaml
├── security/
│   ├── dependency-check-suppressions.xml
│   ├── .semgrepignore
│   ├── .checkov.yaml
│   ├── .trivyignore
│   └── zap/rules.tsv
├── scripts/
│   ├── dd-upload.sh
│   └── quality_gate.py
├── defectdojo/README.md
├── docs/diagramas/
├── Dockerfile
├── package.json
└── (resto del código de Juice Shop)
```

Pasos de preparación del repo:

1. Clonar `juice-shop/juice-shop`, hacer checkout del último tag estable y anotar la versión en el informe.
2. Copiar el contenido (sin `.git`) al repo nuevo y **borrar `.github/` original** para que no se ejecuten sus workflows.
3. Comprobar si trae `package-lock.json`. Si no lo trae, el job de SCA lo genera con `npm install --package-lock-only --ignore-scripts`, porque Dependency-Check lo necesita para resolver el árbol de dependencias.
4. Activar Actions con permisos `contents: read`, `security-events: write` y `packages: write` (este último solo para el job de despliegue).
5. Proteger `main`: exigir que el check `security-gate` pase antes de fusionar un pull request. Así el gate bloqueante también bloquea el merge, no solo el despliegue.

DefectDojo: se despliega con Docker Compose desde `DefectDojo/django-DefectDojo` (`docker compose up -d`, interfaz en el puerto 8080). Necesita unos 4 GB de RAM libres y unos minutos en el primer arranque; la contraseña de `admin` sale en los logs del contenedor `initializer`. Como los runners de GitHub están en Internet, el puerto se publica con **Cloudflare Tunnel** (`cloudflared tunnel --url http://localhost:8080`). El túnel rápido cambia de URL en cada arranque, así que hay que actualizar el secreto `DD_URL` cada vez que se reinicie; si molesta, se crea un túnel con nombre o se usa una VM gratuita en la nube.

Alternativa descartada: levantar DefectDojo dentro del propio pipeline. Sería efímero, se perdería el histórico y no habría deduplicación entre ejecuciones, que es justo lo que se evalúa en el objetivo 5.

Secretos del repositorio (Settings → Secrets and variables → Actions; nunca en el código):

Secreto	Para qué	De dónde sale
`DD_URL`	URL base de DefectDojo	URL del túnel
`DD_API_KEY`	Token de la API v2	DefectDojo → menú de usuario → API v2 Key (mejor de un usuario `ci-bot` con rol Writer, no de `admin`)
`NVD_API_KEY`	Descarga de la base NVD por Dependency-Check	Solicitud gratuita en nvd.nist.gov; sin ella la primera descarga tarda muchísimo o falla
`DISCORD_WEBHOOK` (opcional)	Aviso al equipo cuando falla un gate	Webhook de un canal del grupo

`GITHUB_TOKEN` lo da Actions automáticamente y sirve para publicar la imagen en GHCR.

4. Pipeline etapa a etapa y quality gates

Un único workflow (`devsecops.yml`) con 8 jobs que se lanza en cada push y pull request a `main`. Los análisis de la etapa de build van en modo aviso para que el DAST llegue a ejecutarse; el DAST y el gate final de release bloquean de verdad y, si fallan, no hay despliegue.

Patrón común de cada job de seguridad (es lo que hace que el diagrama 1.a y el YAML coincidan uno a uno):

1. Ejecutar la herramienta **sin que falle por hallazgos** (exit 0) y generar el informe.

2. Enviar el informe a DefectDojo con `scripts/dd-upload.sh` (`if: always()`, así llega aunque algo falle).
3. Guardar el informe como artefacto del run (evidencia y entrada del gate final).
4. Evaluar el quality gate con `scripts/quality_gate.py --tool <x> --report <fichero> --threshold <umbral>`: escribe una tabla en el resumen del job y devuelve 1 si se supera el umbral. En modo aviso ese paso lleva `continue-on-error: true` y deja una anotación `::warning`.

Así el umbral vive en un único sitio (nuestro script), es igual para todas las herramientas y se puede justificar en el informe sin depender de los flags de cada una.

#	Job	Etapa	Qué hace	Depende de
1	`build`	Build	`docker build` de Juice Shop, guarda la imagen como artefacto `image.tar`	—
2	`sca`	Build	Dependency-Check sobre `package.json` / `package-lock.json`	—
3	`sast`	Build	Semgrep sobre el código (`routes/`, `lib/`, `frontend/src/`…)	—
4	`iac`	Build	Checkov sobre `Dockerfile` y `k8s/`	—
5	`container`	Build	Trivy sobre `image.tar`	`build`
6	`dast`	Pruebas	Arranca la imagen en `localhost:3000` y lanza ZAP full scan con ajax spider	`build`
7	`security-gate`	Release	Gate consolidado sobre los informes de 2–5	2, 3, 4, 5, 6 (`if: always()`)
8	`deploy`	Despliegue	Publica la imagen en GHCR y la despliega en un clúster `kind` (smoke test)	`dast` y `security-gate`

Además, un job `notify` con `if: failure()` avisa por Discord (opcional) con el enlace al run; GitHub ya envía correo al autor del commit cuando falla un workflow.

Quality gates:

Gate	Job	Umbral	Modo	Justificación
G1	`sca`	⩾ 1 dependencia con CVSS ⩾ 7.0 (alta o crítica)	Aviso	Juice Shop trae dependencias vulnerables a propósito; bloquear aquí impediría llegar al DAST
G2	`sast`	⩾ 1 hallazgo de severidad `ERROR`	Aviso	Mismo motivo; los `WARNING` e `INFO` solo se registran
G3	`iac`	⩾ 1 check fallido	Aviso	Checkov sin cuenta de Prisma Cloud no asigna severidad, así que el umbral es por número de fallos
G4	`container`	⩾ 1 vulnerabilidad `CRITICAL` con parche disponible	Aviso	Se reevalúa de forma bloqueante en G6
G5	`dast`	⩾ 1 alerta de riesgo `High`	**Bloqueante**	Una alerta alta en ejecución es explotable desde fuera; no puede llegar a producción
G6	`security-gate`	CVSS ⩾ 9.0 en SCA **o** ⩾ 1 `CRITICAL` en Trivy	**Bloqueante**	Es el gate de release: lo que no se podía parar antes del DAST se para antes del despliegue y del merge

G6 fallará con seguridad en Juice Shop (lleva dependencias con CVE críticos conocidos), así que garantiza la evidencia de «ejecución que falla por un quality gate». Si ZAP no encuentra ninguna alerta `High`, G5 pasa y se documenta tal cual; no se baja el umbral para forzar el fallo.

Demostrar que el gate reacciona en los dos sentidos (lo valora el objetivo 4): en una rama `demo/gate` se añaden a `security/.trivyignore` y al fichero de supresiones de Dependency-Check los CVE que disparan G6, marcados como riesgo aceptado con motivo y fecha. El run de esa rama pasa G6 y despliega, y el de `main` sigue fallando. Las dos ejecuciones van a evidencias.

Esqueleto del workflow (base para Rol A; cada rol completa su job):

```
name: DevSecOps

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

env:
  IMAGE: juice-shop-st20
  DD_PRODUCT: Juice Shop ST20
  DD_ENGAGEMENT: CI/CD GitHub Actions

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t $IMAGE:${{ github.sha }} .
      - run: docker save $IMAGE:${{ github.sha }} -o image.tar
      - uses: actions/upload-artifact@v4
        with:
          name: image
          path: image.tar

  sca:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: test -f package-lock.json || npm install --package-lock-only --
ignore-scripts
      - uses: dependency-check/Dependency-Check_Action@main
        with:
          project: juice-shop
          path: .
          format: XML
          out: reports
          args: >
            --format HTML
            --nvdApiKey ${{ secrets.NVD_API_KEY }}
            --suppression security/dependency-check-suppressions.xml
      - if: always()
```

```
        run: scripts/dd-upload.sh "Dependency Check Scan" reports/dependency-
check-report.xml "SCA - Dependency-Check"
        env:
          DD_URL: ${{ secrets.DD_URL }}
          DD_API_KEY: ${{ secrets.DD_API_KEY }}
      - if: always()
        uses: actions/upload-artifact@v4
        with:
          name: report-sca
          path: reports/
      - name: Quality gate G1
        continue-on-error: true
        run: python3 scripts/quality_gate.py --tool dependency-check --report
reports/dependency-check-report.xml --threshold 7.0 --mode warn

  sast:
    runs-on: ubuntu-latest
    container: semgrep/semgrep
    steps:
      - uses: actions/checkout@v4
      - run: semgrep scan --config p/owasp-top-ten --config p/nodejsscan --
config p/secrets --json --output semgrep.json

  iac:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: bridgecrewio/checkov-action@v12
        with:
          directory: .
          framework: dockerfile,kubernetes
          output_format: json
          output_file_path: reports
          soft_fail: true
          config_file: security/.checkov.yaml

  container:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          name: image
      - uses: aquasecurity/trivy-action@0.28.0
        with:
```

```
          input: image.tar
          format: json
          output: trivy.json
          trivyignores: security/.trivyignore
          exit-code: 0

  dast:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          name: image
      - run: docker load -i image.tar
      - run: docker run -d --name app -p 3000:3000 $IMAGE:${{ github.sha }}
      - run: timeout 180 bash -c 'until curl -sf http://localhost:3000 >
/dev/null; do sleep 5; done'
      - uses: zaproxy/action-full-scan@v0.12.0
        with:
          target: http://localhost:3000
          rules_file_name: security/zap/rules.tsv
          cmd_options: -a -j -x report_xml.xml
          allow_issue_writing: false
          fail_action: false
      - name: Quality gate G5
        run: python3 scripts/quality_gate.py --tool zap --report
report_json.json --threshold High --ignore security/zap/rules.tsv

  security-gate:
    needs: [sca, sast, iac, container, dast]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          pattern: report-*
          path: reports
      - run: python3 scripts/quality_gate.py --tool dependency-check --report
reports/report-sca/dependency-check-report.xml --threshold 9.0
      - run: python3 scripts/quality_gate.py --tool trivy --report
reports/report-container/trivy.json --threshold CRITICAL

  deploy:
    needs: [dast, security-gate]
```

```
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: helm/kind-action@v1
      - run: kubectl apply -f k8s/

  notify:
    needs: [security-gate, dast, deploy]
    if: failure()
    runs-on: ubuntu-latest
    steps:
      - run: echo "Pipeline bloqueado por un quality gate"
```

En el esqueleto solo `sca` está completo como modelo: el resto de jobs repite los pasos 2–4 del patrón (subida, artefacto `report-<job>`, gate). Las versiones de las acciones se comprueban y se fijan al montarlo; las de arriba son orientativas.

5. Diagramas del objetivo 1

Estos son los dos diagramas de diseño (versión 1); en el informe se redibujan en draw.io con el mismo contenido y se actualizan si cambia la implementación. Cada caja del 1.a es un job de `devsecops.yml` y cada gate está en la tabla de la sección 4.

1.a Pipeline DevSecOps

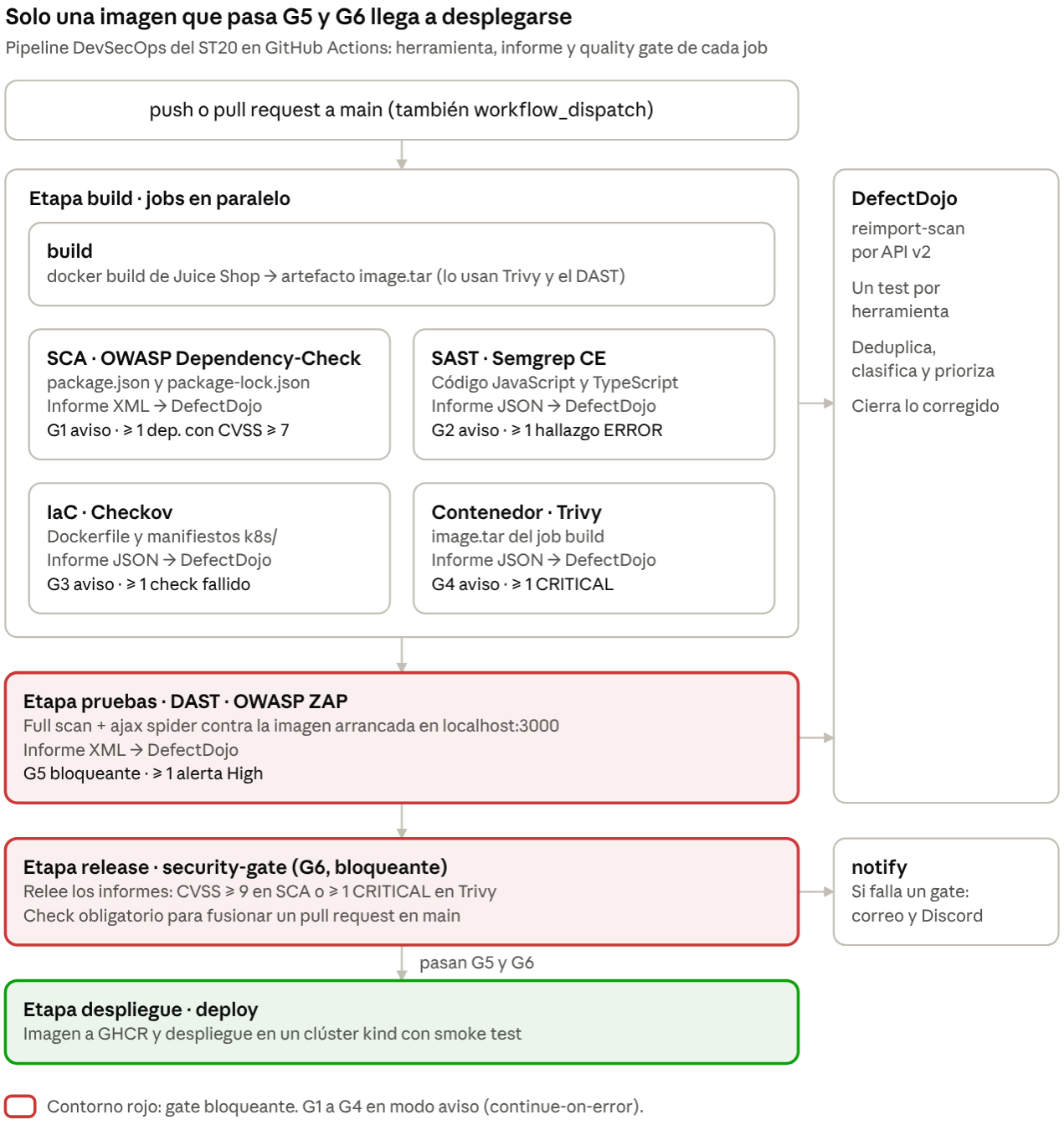


Diagrama 1.a · pipeline DevSecOps · 8 jobs, 6 quality gates

Los cuatro análisis de build corren en paralelo y solo avisan; el DAST y el gate de release son los que pueden cortar el despliegue, y todos los informes acaban en DefectDojo.

1.b Flujo ante una vulnerabilidad

Todo hallazgo se registra; solo un gate bloqueante para el pipeline

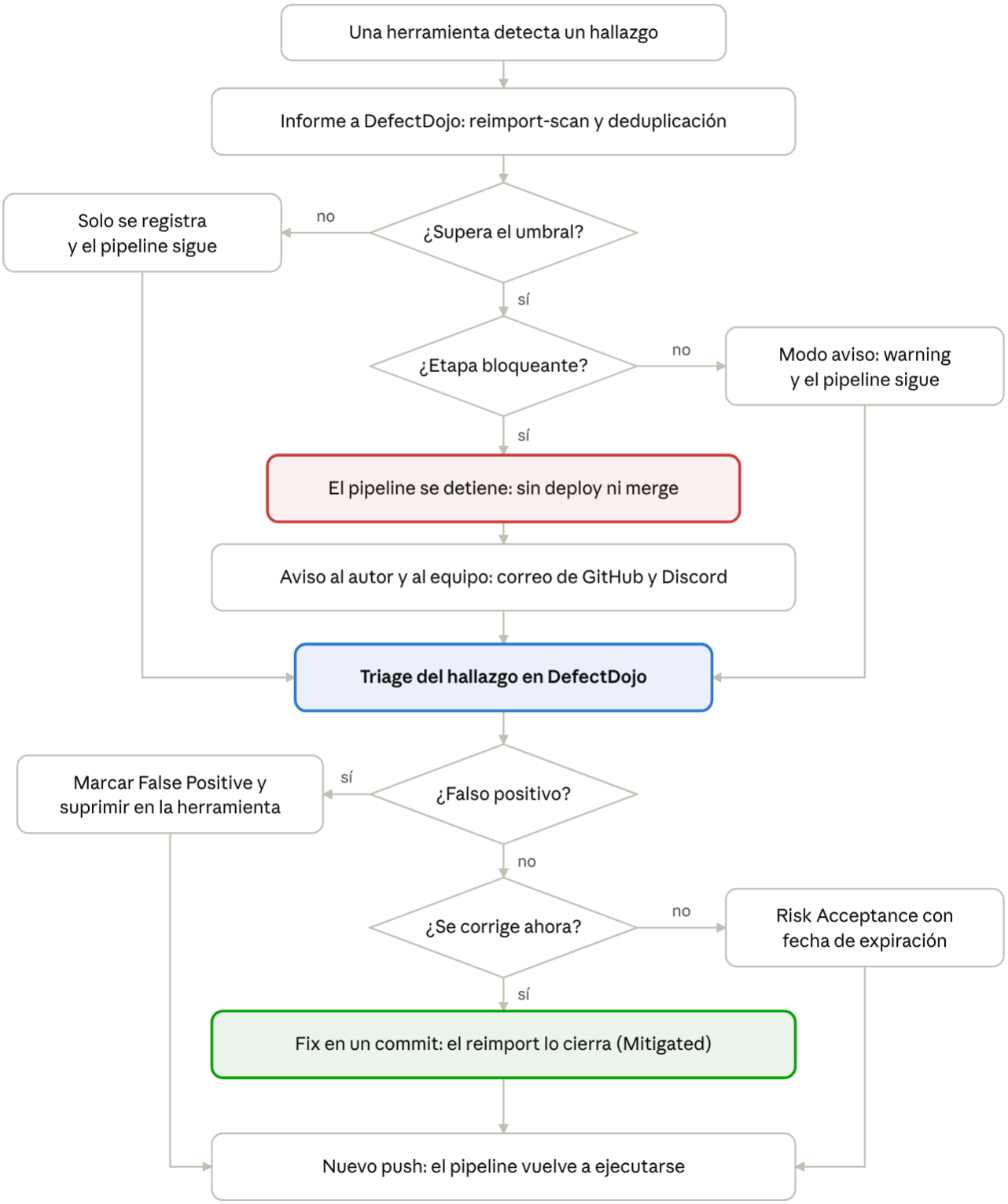


Diagrama 1.b · flujo ante una vulnerabilidad · 4 decisiones

Cualquier hallazgo queda registrado y pasa por triage en DefectDojo, supere o no el umbral; solo los gates bloqueantes detienen el pipeline y disparan el aviso, y todo cierre se confirma con un nuevo push.

6. DefectDojo: integración y gestión de hallazgos

Cada job sube su informe con `reimport-scan` de la API v2 al mismo producto y engagement; DefectDojo crea el contexto la primera vez, deduplica entre herramientas y entre ejecuciones, y cierra solo lo que ya no aparece.

Estructura en DefectDojo:

Nivel	Valor
Product Type	SSII
Product	Juice Shop ST20
Engagement	CI/CD GitHub Actions (tipo CI/CD, enlazado al repo)
Tests (uno por herramienta)	SCA - Dependency-Check · SAST - Semgrep · IaC - Checkov · Container - Trivy · DAST - ZAP

Por qué `reimport-scan` y no `import-scan`: `import-scan` crea un test nuevo en cada push y en dos semanas habría cientos de tests duplicados. `reimport-scan` actualiza el test existente: marca hallazgos nuevos, reactiva los que vuelven y, con `close_old_findings=true`, cierra como mitigados los que desaparecen tras un arreglo.

`scripts/dd-upload.sh` (lo usa cada job; recibe tipo de escaneo, fichero y título del test):

```
#!/usr/bin/env bash
set -euo pipefail

scan_type="$1"
report="$2"
test_title="$3"

curl -sS --fail-with-body -X POST "$DD_URL/api/v2/reimport-scan/" \
  -H "Authorization: Token $DD_API_KEY" \
  -F "scan_type=$scan_type" \
  -F "file=@$report" \
  -F "test_title=$test_title" \
  -F "product_type_name=SSII" \
  -F "product_name=$DD_PRODUCT" \
```

```
  -F "engagement_name=$DD_ENGAGEMENT" \
  -F "auto_create_context=true" \
  -F "deduplication_on_engagement=true" \
  -F "close_old_findings=true" \
  -F "minimum_severity=Info" \
  -F "active=true" \
  -F "verified=false" \
  -F "build_id=$GITHUB_RUN_ID" \
  -F "commit_hash=$GITHUB_SHA" \
  -F "branch_tag=$GITHUB_REF_NAME"
```

Configuración necesaria en DefectDojo (una vez):

1. System Settings → activar *Deduplicate findings* y *False positive history* (un hallazgo marcado como falso positivo no vuelve a aparecer como activo en el siguiente reimport).
2. Crear el usuario `ci-bot` con rol Writer sobre el producto y usar su API key, no la de `admin`.
3. Definir SLA por severidad (p. ej. Critical 7 días, High 30) para que la priorización tenga fechas en el informe.
4. Activar notificaciones de *Scan added* para el equipo.

Triage (lo que se hace a mano tras cada run relevante):

Decisión	En DefectDojo	En el repo
Verdadero positivo	Marcar *Verified*, asignar responsable, queda con SLA	Se corrige en un commit; el siguiente reimport lo cierra como *Mitigated*
Falso positivo	Marcar *False Positive* con justificación	Suprimirlo en la herramienta (`nosemgrep`, `.trivyignore`, supresiones de Dependency-Check, `IGNORE` en `rules.tsv` de ZAP, `skip-check` de Checkov) para que no cuente en el gate
Riesgo aceptado	*Risk Acceptance* con motivo y fecha de expiración	Igual que el falso positivo, con comentario y fecha
Duplicado	Lo detecta solo (mismo CVE desde Dependency-Check y Trivy, por ejemplo)	Nada

La supresión en el repo es necesaria porque el gate evalúa los informes del run, no el estado de DefectDojo. Mejora opcional: que `quality_gate.py` consulte en la API los hallazgos activos, no falsos positivos y sin riesgo aceptado del engagement, y decida con eso; así el triage en DefectDojo gobierna directamente el pipeline.

7. Evidencias, informe, entrega y calendario

El trabajo se cierra en tres bloques que coinciden con los seguimientos: diseño + plataforma para el 17/18, pipeline completo fallando en G6 para el 24/25, y triage + informe para el 2 de diciembre.

Calendario:

1. **Hasta el 17/18 nov (seguimiento 1):** diagramas 1.a y 1.b en versión 1, repo creado con Juice Shop, DefectDojo levantado y accesible por el túnel, jobs `build` y `sca` funcionando y subiendo a DefectDojo.
2. **Hasta el 24/25 nov (seguimiento 2):** los 8 jobs integrados, todas las herramientas con al menos un hallazgo en DefectDojo, G5 y G6 operativos, primer run fallido por gate guardado.
3. **Hasta el 1 dic:** triage en DefectDojo, rama `demo/gate` que pasa y despliega, diagramas actualizados a lo implementado, informe cerrado. El 2 dic solo se revisa y se sube.

Evidencias mínimas (carpeta `evidencias/` del zip; en el informe solo las justas, el resto en anexos):

- [ ] Log completo (descargado de Actions) de un run de `main` que falla en G6, con captura del grafo de jobs
- [ ] Captura del resumen de cada job con la tabla del gate (G1–G6)
- [ ] Al menos un hallazgo concreto por herramienta: Dependency-Check, Semgrep, Checkov, Trivy y ZAP
- [ ] DefectDojo: vista del engagement con los 5 tests, listado de hallazgos por severidad, un duplicado detectado, un falso positivo marcado y una aceptación de riesgo
- [ ] Dos reimports seguidos del mismo test mostrando que no se duplican hallazgos
- [ ] Run de la rama `demo/gate` que pasa G6 y despliega en `kind`
- [ ] Captura de los secretos configurados en GitHub (solo los nombres)
- [ ] Aviso recibido (correo de GitHub o Discord) del run fallido

Estructura del informe (≤ 15 páginas sin portada ni anexos):

Apartado	Pág. aprox.	Responsable
1. Introducción, contexto y enlace al repositorio	1	Rol A
2. Aplicación y herramientas seleccionadas, con justificación	2	Rol B
3. Diseño del pipeline: diagramas 1.a y 1.b y cambios respecto al diseño inicial	3	Rol A (1.a) y Rol C (1.b)
4. Integración de cada herramienta y quality gates	3	Rol B (G1–G4) y Rol C (G5–G6)
5. Resultados por herramienta	3	Cada rol, sus herramientas
6. Gestión en DefectDojo	2	Rol C
7. Conclusiones y respuesta a la pregunta del cliente	1	Rol C, revisa todo el equipo
Anexos: solo evidencias	—	Rol C reúne

El apartado 7 debe responder explícitamente al cliente: sí, los incidentes se explican por su forma de trabajar, porque ninguna etapa del pipeline verificaba código, dependencias ni configuración, y con microservicios en Kubernetes eso multiplica la superficie de ataque.

Contenido de PAI4-ST20.zip:

```
PAI4-ST20.zip
├── PAI4-ST20-Informe.pdf
├── codigo/
│   ├── .github/workflows/devsecops.yml
│   ├── .github/dependabot.yml
│   ├── k8s/
│   ├── security/
│   ├── scripts/
│   └── defectdojo/README.md
└── evidencias/
    ├── logs/
    ├── informes/
    └── capturas/
```

No se incluye el código de Juice Shop (está en el repositorio enlazado). Antes de comprimir: `git grep -nI -E "api[_-]?key|token|password" -- . ':!node_modules'` sobre nuestros ficheros para confirmar que no hay credenciales.